CS336 作业 2（系统）：系统与并行性 版本 1.0.4

2025 年春季

1 作业概述

在这个作业中，你将获得一些关于提高单 GPU 训练速度和扩展训练到多个 GPU 的实践经验。

你将实现什么。

1. 基准测试和性能分析框架
2. Flash Attention 2 Triton 内核
3. 分布式数据并行训练
4. 优化器状态分片

What the code looks like. 所有分配代码以及这份文档都可在 GitHub 上获取：

`github.com/stanford-cs336/assignment2-systems`

请 `git clone` 仓库。如果有任何更新，我们会通知您，您可以 `git pull` 以获取最新版本。

1. `cs336-basics/`：在本作业中，您将分析我们在第一次作业中构建的一些组件。此文件夹包含第一次作业的参考代码，因此您会在这里找到一个 `cs336-basics/pyproject.toml` 和一个 `cs336-basics/cs336_basics/*` 模块。如果您想使用自己的模型实现，可以修改基础目录中的 `pyproject.toml` 文件，使其指向您自己的包。
2. `/`: cs336-systems 基础目录。我们创建了一个名为 `cs336_systems` 的空模块。请注意这里没有代码，因此你应该能够从零开始做任何事。
3. `tests/*.py`: 这里包含所有你必须通过的测试。这些测试调用了定义在 `tests/adapters.py` 中的钩子。你需要实现适配器来将你的代码连接到这些测试。编写更多测试和/或修改测试代码有助于调试你的代码，但你的实现需要通过最初提供的测试套件。
4. `README.md`: 这个文件包含有关预期目录结构的更多详细信息，以及一些关于设置你环境的基本说明。

如何提交。你将向 Gradescope 提交以下文件：

- `writeup.pdf`: 回答所有书面问题。请排版你的回答。
- `code.zip`: 包含你编写的所有代码。

在 `test_and_make_submission.sh` 中运行脚本以创建 `code.zip` 文件。

在作业的第一部分，我们将研究如何优化 Transformer 模型的性能，以最有效地利用 GPU。我们将分析模型以了解其在前向和后向传递中花费的时间和内存，然后通过自定义 GPU 内核优化自注意力操作，使其比直接的 PyTorch 实现更快。在作业的后续部分，我们将利用多个 GPU。

1.1 性能分析和基准测试

在实施任何优化之前，首先对程序进行性能分析以了解其资源消耗情况（例如时间和内存）是有帮助的。否则，我们可能会优化模型中那些未考虑显著时间或内存的部分，从而无法看到可衡量的端到端改进。

我们将实现三条性能评估路径：(a) 使用 Python 标准库进行简单的端到端基准测试，以测量我们的前向和后向传递时间；(b) 使用 NVIDIA Nsight Systems 工具分析时间如何在 CPU 和 GPU 上的操作中分布；(c) 分析内存使用情况。

1.1.1 设置 - 导入你的 `Basics` Transformer 模型

让我们先确保你可以从上一个作业中加载模型。在之前的作业中，我们将模型设置在一个 Python 包中，以便后续可以轻松导入。我们在 `./cs336-basics` 文件夹中添加了模型的实现代码，并在 `pyproject.toml` 文件中指向了它。通过调用 `uv run [command]`， `uv` 将自动定位这个本地 `cs336-basics` 包。如果你希望使用自己的模型实现，可以修改 `pyproject.toml` 文件以指向你自己的包。

您可以通过以下方式测试是否可以导入您的模型：

```
~$ uv run python
Using CPython 3.12.10
Creating virtual environment at: /path/to/uv/env/dir
      Built cs336-systems @ file:///path/to/systems/dir
      Built cs336-basics @ file:///path/to/basics/dir
Installed 85 packages in 711ms
Python 3.12.10 (main, Apr  9 2025, 04:03:51) [Clang 20.1.0 ] on linux
...
>>> import cs336_basics
>>>
```

现在应该可以导入作业1的相关模块（例如，对于 `model.py`，您可以使用 `import cs336_basics.model` 来导入）。

1.1.2 模型尺寸

在整个作业中，我们将对模型进行基准测试和性能分析，以更好地理解其性能。为了了解规模变化时的情况，我们将使用和参考以下模型配置。对于所有模型，我们将使用10,000的词汇量和4的批处理大小，并使用不同的上下文长度。这个作业（以及后续的作业）将需要大量结果以表格形式呈现。我们强烈建议您在代码中自动构建您的报告表格，因为使用LaTeX或Markdown格式化表格可能非常繁琐。请参考 `pandas.DataFrame.to_latex()` 和`pandas.DataFrame.to_markdown()`，或者编写自己的函数，从您喜欢的表格表示形式生成它们。

Size	d_model	d_ff	num_layers	num_heads
小	768	3072	12	12
中	1024	4096	24	16
大	1280	5120	36	20
xl	1600	6400	48	25
2.7B	2560	10240	32	32

表1：不同型号规格

1.1.3 端到端基准测试

我们现在将实现一个简单的性能评估脚本。我们将测试我们模型的许多变体（改变精度、交换层等），因此通过命令行参数启用这些变体将非常有价值，以便以后轻松运行。我们还强烈建议使用sbatch或Slurm上的submitit运行基准测试超参数的扫描，例如模型大小、上下文长度等，以便快速迭代。

首先，让我们通过计时前向和后向传递来对模型进行最简单的性能分析。由于我们只测量速度和内存，我们将使用随机权重和数据。

性能测量很微妙——一些常见的陷阱会导致我们无法测量我们想要的内容。在基准测试 GPU 代码时，一个需要注意的问题是 CUDA 调用是异步的。当你调用 CUDA 内核，例如调用 `torch.matmul` 时，函数调用会返回控制权给你的代码，而不会等待矩阵乘法完成。这样，CPU 可以在 GPU 计算 matrix multiplication 的同时继续运行。另一方面，这意味着，天真地测量 `torch.matmul` 调用返回所需的时间并不能告诉我们 GPU 实际运行 matrix multiplication 所需的时间。在 PyTorch 中，我们可以调用 `torch.cuda.synchronize()` 来等待所有 GPU 内核完成，从而让我们能够获得更准确的 CUDA 内核运行时测量。有了这个前提，让我们编写我们的基本性能分析基础设施。

问题 (`benchmarking_script`): 4分

(a) 编写一个脚本来执行模型前向和后向传递的基本端到端基准测试。具体来说，你的脚本应该支持以下功能：

- 给定超参数（例如层数），初始化模型。
- 生成一批随机数据。
- 运行 w 预热步骤（在开始计时之前），然后计时 n 步骤的执行（根据参数决定是仅前向传递，还是前向和后向传递）。对于计时，可以使用 Python `timeit` 模块（例如，使用 `timeit` 函数，或者使用 `timeit.default_timer()`，它提供系统最高分辨率的时钟，因此比 `time.time()`）更适合作为基准测试的默认选项）。
- 在每一步调用 `torch.cuda.synchronize()`。

可交付成果：一个脚本，用于使用给定的超参数初始化一个 `basics` Transformer 模型，创建一个随机数据批次，并计时前向和后向传递。

(b) 对 §1.1.2 中描述的模型大小计时前向和后向传递。使用 5 个预热步骤，并计算 10 次测量步骤的计时平均值和标准差。前向传递需要多长时间？后向传递呢？测量值之间是否存在高变异性，还是标准差很小？

，或标准差很小？

可交付成果：一个包含 1-2 句话的响应，其中包含你的计时结果。

(c) 基准测试的一个问题是未执行预热步骤。不执行预热步骤重复你的分析。这如何影响你的结果？你认为为什么会这样？另外尝试用 1 或 2 个预热步骤运行脚本。为什么结果可能仍然不同？

交付物：2-3 句话的回应。

1.1.4 NsightSystems Profiler

端到端基准测试不能告诉我们我们的模型在正向和反向传递期间花费了哪些时间以及内存，因此无法暴露具体的优化机会。要知道我们的程序在每个组件（例如，函数）上花费了多少时间，我们可以使用分析器。执行分析器通过在函数开始和结束时插入守卫来对代码进行注入，因此可以在函数级别提供详细的执行统计信息（例如，调用次数、平均耗时、该函数累计耗时等）。

标准的 Python 分析器（例如， `CProfile`）无法分析 CUDA 内核，因为这些内核在 GPU 上异步执行。幸运的是，NVIDIA 提供了一个我们可以通过 CLI `nsys` 使用的分析器，我们已经为你安装好了。在这个作业的这一部分，你将使用 `nsys` 来分析你的 Transformer 模型的运行时。使用 `nsys` 非常简单：我们可以简单地用 `nsys profile` 预先运行你上一节的 Python 脚本。例如，你可以分析脚本`benchmark.py`并将输出写入文件 `result.nsys.rep` ，如下所示：

```
~$ uv run nsys profile -o result python benchmark.py
```

然后，您可以使用NVIDIA Nsight Systems桌面应用程序在本地计算机上查看配置文件。在 `CUDA API` 行中选择特定的CUDA API调用（在CPU上）时， `CUDA HW` 行中会突出显示所有相应的内核执行（在GPU上）。

我们鼓励您尝试使用 `nsys profile` 的各种命令行选项来了解它能够做什么。值得注意的是，您可以使用 `--python-backtrace=cuda` 获取每个 CUDA API 调用的 Python 回溯，但这可能会引入开销。您还可以使用 NVTX 范围标注您的代码，这些范围将在捕获所有 CUDA API 调用和相关内核执行的 `NVTX` 行的剖析中显示为块。特别是，您应该使用 NVTX 范围来忽略您的基准测试脚本中的预热步骤（通过在剖析的 `NVTX` 行上应用过滤器）。您还可以隔离负责模型前向和后向传递的哪些内核，甚至可以通过以下方式标注您的实现来隔离负责自注意力层不同部分的哪些内核：

```python
...
import torch.cuda.nvtx as nvtx

@nvtx.range("scaled dot product attention")
def annotated_scaled_dot_product_attention(
    ... # Q, K, V, mask
)
    ...
    with nvtx.range("computing attention scores"):
        ... # compute attention scores between Q and K

    with nvtx.range("computing softmax")
        ... # compute softmax of attention scores

    with nvtx.range("final matmul")
        ... # compute output projection
```

```

    return ...
```

您可以通过

```
cs336_basics.model.scaled_dot_product_attention = annotated_scaled_dot_product_attention
```

在您的基准测试脚本中交换您的原始实现与标注版本。最后，您可以使用 `--pytorch` 命令行选项与 `nsys` 自动使用 NVTX 范围标注对 PyTorch C++ API 的调用。

问题 (nsys_profile)：5分

使用 `nsys` 对每个模型大小（如表1所述）以及上下文长度为128、256、512和1024的前向传递、反向传递和优化器步骤进行性能分析（对于较大的模型，某些上下文长度可能会耗尽内存，在这种情况下，请在报告中注明）。

(a) 您的前向传递总共花费了多长时间？这与我们之前测量的一致吗
使用Python标准库？
可交付成果：1-2句回复。

(b) 在前向传递期间，哪个CUDA内核占用了最多的累积GPU时间？在您的模型的单次前向传递中，该内核被调用了多少次？当您同时进行前向和反向传递时，这是占用最多运行时间的同一个内核吗？（提示：查看“统计系统视图”下的“CUDA GPU内核摘要”，并使用NVTX范围进行筛选，以识别模型的哪些部分负责哪些内核。）

成果：1-2句简短回答。

(c) 虽然绝大多数FLOPs发生在矩阵乘法中，但你可能会注意到其他几个内核仍然占用了相当大的整体运行时间。除了矩阵乘法之外，你还看到哪些内核在正向传递中占用了非平凡的CUDA运行时间？

成果：1-2句简短回答。

(d) 使用你的AdamW实现运行一个完整的训练步骤（即正向传递、计算损失、运行反向传递，最后进行优化器步骤，就像训练期间会做的那样）。与仅进行推理（仅正向传递）相比，矩阵乘法所花费的时间比例如何变化？其他内核呢？

成果：1-2句简短回答。

(e) 比较你的模型在正向传递期间自注意力层中softmax操作与矩阵乘法操作的运行时间。运行时间的差异与FLOPs的差异相比如何？

可交付成果：1-2句回复。

1.1.5 混合精度

到目前为止，在这个作业中，我们一直使用FP32精度运行——所有模型参数和激活值都使用 `torch.float32` 数据类型。然而，现代NVIDIA GPU包含专门用于在较低精度下加速矩阵乘法的GPU核心（张量核心）。例如，NVIDIA A100的规格表表明，其使用FP32时的最大吞吐量为19.5 TFLOP/秒，而使用FP16（半精度浮点数）或BF16（脑浮点数）时的最大吞吐量要显著高得多，达到312 TFLOP/秒。因此，使用较低精度的数据类型应该能帮助我们加快训练和推理速度。

然而，将我们的模型简单转换为低精度格式可能会降低模型精度。例如，在实践中，许多梯度值往往太小而无法用FP16表示，因此在用FP16精度简单训练时就会变为零。为了解决这个问题，在用FP16训练时通常使用损失缩放——损失简单地乘以一个缩放因子，增加梯度幅度，以防止它们归零。此外，FP16的动态范围低于FP32，这可能导致溢出，表现为NaN损失。全bfloat16训练通常更稳定（因为BF16的动态范围与FP32相同），但与FP32相比，仍可能影响最终模型性能。

要利用低精度数据类型的加速效果，通常使用混合精度训练。在 PyTorch 中，这是通过 `torch.autocast` 上下文管理器实现的。在这种情况下，某些操作（例如矩阵乘法）会使用低精度数据类型执行，而其他需要完整 FP32 动态范围的操作（例如累加和缩减）则保持不变。例如，以下代码会在前向传播期间自动识别哪些操作需要在低精度执行，并将这些操作转换为指定的数据类型：

```
model : torch.nn.Module = ... # e.g. your Transformer model
dtype : torch.dtype = ... # e.g. torch.float16
x : torch.Tensor = ... # input data

with torch.autocast(device="cuda",dtype=dtype):
    y = model(x)
```

如前所述，即使被累加的张量已经被向下转型，通常也建议保持累加操作在更高精度。以下练习将帮助你理解为什么这是这种情况。

Problem (mixed_precision_accumulation): 1 point

Run the following code and commment on the (accuracy of the) results.

```
s = torch.tensor(0,dtype=torch.float32)
for i in range(1000):
    s += torch.tensor(0.01,dtype=torch.float32)
print(s)

s = torch.tensor(0,dtype=torch.float16)
for i in range(1000):
    s += torch.tensor(0.01,dtype=torch.float16)
print(s)

s = torch.tensor(0,dtype=torch.float32)
for i in range(1000):
    s += torch.tensor(0.01,dtype=torch.float16)
print(s)

s = torch.tensor(0,dtype=torch.float32)
for i in range(1000):
    x = torch.tensor(0.01,dtype=torch.float16)
    s += x.type(torch.float32)
print(s)
```

Deliverable: A 2-3 sentence response.

我们将首先针对一个玩具模型进行混合精度应用以获得直观理解，然后将其应用于我们的基准测试脚本。

问题 (benchmarking_mixed_precision)：2 分

(a) 考虑以下模型：

```python
class ToyModel(nn.Module):
    def __init__(self, in_features: int, out_features: int):
        super().__init__()
        self.fc1 = nn.Linear(in_features, 10, bias=False)
        self.ln = nn.LayerNorm(10)
        self.fc2 = nn.Linear(10, out_features, bias=False)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.ln(x)
        x = self.fc2(x)
        return x
```

假设我们在GPU上训练模型，且模型参数最初为FP32。我们希望使用FP16的自动混合精度。以下的数据类型是什么：

- 自回归上下文中的模型参数，
- 第一个前馈层的输出 (`ToyModel.fc1`)，
- 层归一化的输出 (`ToyModel.ln`)，
- 模型的预测 logits，
- 损失 ，
- 以及模型的梯度？

交付物：上述每个组件的数据类型。

(b) 你应该已经注意到，FP16混合精度自动转换对层归一化层与前馈层处理方式不同。层归一化哪些部分对混合精度敏感？如果我们使用BF16代替FP16，是否仍然需要以不同方式处理层归一化？为什么或为什么不？

交付物：2-3句回答。

(c) 修改你的基准测试脚本，使其可选地使用BF16混合精度运行模型。
对 §1.1.2 中描述的每种语言模型大小，分别计时前向和后向传播（使用和不使用混合精度）。比较使用全精度与混合精度的结果，并评论随着模型大小变化时出现的任何趋势。您可能会发现 `nullcontext` no-op 上下文管理器很有用。

交付物：一个包含您的计时和评论的 2-3 句话的响应。

1.1.6 分析内存

到目前为止，我们一直在关注计算性能。现在，我们将注意力转向内存，内存是语言模型训练和推理中的另一个主要资源。PyTorch 还附带了一个强大的内存分析器，可以跟踪随时间的内存分配情况。

要使用内存分析器，您可以按如下方式修改您的基准测试脚本：

```python

... # warm-up phase in your benchmarking script

# Start recording memory history.
torch.cuda.memory._record_memory_history(max_entries=1000000)

... # what you want to profile in your benchmarking script

# Save a pickle file to be loaded by PyTorch's online tool.
torch.cuda.memory._dump_snapshot("memory_snapshot.pickle")

# Stop recording history.
torch.cuda.memory._record_memory_history(enabled=None)
```

这将输出一个文件 `memory_snapshot.pickle`，您可以将其加载到以下在线工具 `https://pytorch.org/memory_viz` 中。该工具将允许您查看整体内存使用时间线以及每个单独分配的详细信息，包括其大小和指向原始代码的堆栈跟踪。要使用此工具，您应在 Web 浏览器中打开上面的链接，然后将您的 Pickle 文件拖放到页面上。

现在，您将使用 PyTorch 分析器来分析您的模型的内存使用情况。

问题 (memory_profiling)：4 分

从表1中，使用上下文长度为128、256和512对 `2.7B` 模型的正向传递、反向传递和优化器步骤进行性能分析。

(a) 在您的分析脚本中添加一个选项，以通过内存分析器运行您的模型。这可能有助于重用您之前的一些基础设施（例如，激活混合精度、加载特定模型大小等）。然后，运行您的脚本，获取 `2.7B` 模型在仅进行推理（仅前向传递）或完整训练步骤时的内存分析。您的内存时间线看起来如何？您能否根据看到的峰值判断正在运行哪个阶段？

交付物：两张“活跃内存时间线”的 `2.7B` 模型图像，来自 `memory_viz` 工具：一张用于前向传递，一张用于运行完整训练步骤（前向和反向传递，然后是优化器步骤），以及一段 2-3 句的回应。

(b) 在进行前向传递时，每个上下文长度的峰值内存使用是多少？
在执行完整训练步骤时？
交付物：每个上下文长度包含两个数字的表格。

(c) 查找 `2.7B` 模型在混合精度使用情况下的峰值内存使用量，包括正向
执行通过和完整的优化器步骤。混合精度是否显著影响内存使用？
可交付成果：一段2-3句话的回应。

(d) 考虑 `2.7B` 模型。在我们的参考超参数下，Transformer 残差流中激活张量的大小是多少（以单精度表示）？以 MB 为单位给出这个大小（即，将字节数除以 1024^2）。

可交付成果：用1-2句话说明你的推导过程。

(e) 现在仔细查看 `pytorch.org/memory_viz` 的“活动内存时间线”，这是 `2.7B` 模型执行前向传播时内存快照的一部分。当你降低“细节”级别时，工具会将最小的内存分配隐藏到对应的级别（例如，“细节”设置为10%时仅显示

（前10%的最大分配）。显示的最大分配有多大？查看
堆栈跟踪，你能告诉我这些内存分配来自哪里吗？
交付物：1-2句回复。

1.2 使用FlashAttention-2优化注意力机制

1.2.1 基准测试PyTorch注意力机制

你的性能分析可能表明，你的注意力层在内存和计算方面都有优化的机会。从宏观上看，注意力操作包括矩阵乘法，然后是softmax，接着是另一个矩阵乘法：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\text{mask}\left(\frac{Q^\top K}{\sqrt{d_k}}\right)\right)V \tag{1}$$

原始的注意力实现需要为每个批次/头元素保存形状为 `seq_len` × `seq_len` 的注意力分数矩阵，随着序列长度的增加，这些矩阵可能会变得非常大，导致具有长输入或输出的任何任务出现内存不足错误。我们将根据FlashAttention-2论文实现一个注意力核，该核通过分块计算注意力，避免显式地创建 `seq_len` × `seq_len`注意力分数矩阵，从而能够扩展到更长的序列长度。

问题 (`pytorch_attention`)：2 分

(a) 在不同尺度上对你的注意力实现进行基准测试。编写一个脚本，该脚本将：

(a) 将批处理大小固定为 8，并且不要使用多头注意力（即移除头维度）。
(b) 遍历 [16，32，64，的笛卡尔积 128] for 头嵌入维度128]，以及 for 头嵌入维度 d_{model}，以及 [256，1024，4096，8192，16384] 用于序列长度。
(c) 为适当大小创建随机输入 Q, K, V。
(d) 使用输入对注意力进行100次前向传递。
(e) 在反向传播开始前测量已使用的内存量，并计时100次反向传播。

(f) 确保进行预热，并在每次前向/反向传播后调用 `torch.cuda.synchronize()`。
传播。

报告这些配置的计时（或内存不足错误）。在什么规模下你会遇到内存不足错误？在找到的最小运行内存不足的配置中，如何计算注意力使用的内存（你可以使用作业1中Transformer内存使用方程）。反向传播节省的内存如何随序列长度变化？你会如何消除这种内存开销？

交付物：一个包含你的计时、内存使用计算过程和1-2段回应的表格。

1.3 基准测试JIT编译的注意力

自2.0版本起，PyTorch也附带了一个强大的即时编译器，会自动尝试对PyTorch函数应用多种优化：参见 `https://pytorch.org/tutorials/intermediate/torch_compile_tutorial.html` 了解介绍。特别是，它会通过动态分析你的计算图来自动生成融合的Triton内核。使用PyTorch编译器的接口非常简单。例如，如果我们想将其应用于模型的一个单层，可以使用：

```
layer = SomePyTorchModule(...)
compiled_layer = torch.compile(layer)
```

现在， `compiled_layer` 在功能上表现得就像 `layer` （例如，具有其前向和后向传递）。我们也可以使用 `torch.compile(model)`编译整个PyTorch模型，甚至是一个调用PyTorch操作的Python函数。

问题 (`torch_compile`)：2 分

(a) 将您的注意力基准测试脚本扩展以包含 PyTorch 注意力实现的编译版本，并将其性能与上述 `pytorch_attention` 问题中相同配置的未编译版本进行比较。

交付物：一个表格，比较您的编译注意力模块与上述 `pytorch_attention` 问题中未编译版本的 forward 和 backward 传递时间。

(b) 现在，在您的端到端基准测试脚本中编译您的整个 Transformer 模型。前向传递的性能如何变化？结合前向和 backward 传递以及优化器步骤呢？

交付物：一个比较您的普通和编译 Transformer 模型的表格。

鉴于我们看到的关于序列长度的扩展行为，我们需要显著改进以处理长序列。即使使用 `torch.compile`，当前实现在中长序列长度下也受到非常差的内存访问模式的影响。为此，我们将编写 FlashAttention-2 的 Triton 实现，在那里我们将对如何访问内存以及何时计算什么有显著更多的控制。

1.3.1 示例 - 加权求和

为了介绍您需要了解的 Triton 内容以及它如何与 PyTorch 互操作，我们将通过一个“加权求和”操作的示例内核进行说明。有关快速上手 Triton 的更多资源，请参阅 Triton 的教程。我们注意到这些教程不使用新的、方便的块指针抽象，我们将在下面进行介绍。

给定输入矩阵 X，我们将将其元素乘以列向量的权重 w，并对每一行求和，得到 X 和 w 的矩阵-向量积。我们将首先处理此操作的正向传递，然后编写用于反向传递的 Triton 内核。

正向传递 我们内核的正向传递只是以下广播内积。

```python
def weighted_sum(x, weight):
    # Here, assume that x has n-dim shape [..., D], and weight has 1D shape [D]
    return (weight * x).sum(axis=-1)
```

在编写我们的 Triton 内核时，我们将让每个程序实例（可能并行运行）计算 x 一组行（tile）的加权求和，并将相应的标量输出写入输出张量。在 Triton 中，程序实例是一组运行相同程序的线程块，这些线程块可以在 GPU 上并行运行。我们不是将张量作为参数传递，而是传递指向它们第一个元素的指针，以及每个张量的步长，这些步长告诉我们如何在轴上移动。

我们可以使用步长来加载与运行实例中求和的 x 行块对应的张量，并使用程序 ID 来分配工作（即，实例 i 将处理 x 的第 i 行块）。在 Triton 和 PyTorch 的简单情况下，前向传递的主要区别是需要进行指针运算和显式的加载/存储。我们将使用块指针抽象与 x 来大大简化指针运算，但这意味着我们需要进行一些设置来准备块指针。

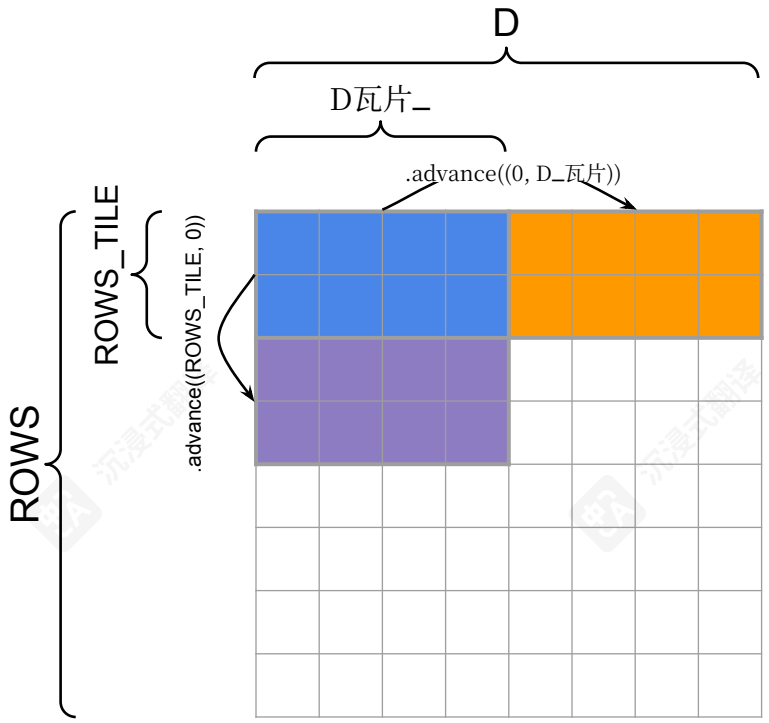

图1：在加权求和核示例（第1.3.1节）中瓦片和推进块指针

`tl.make_block_ptr` 来大大简化指针运算，尽管这意味着我们需要进行一些设置来准备块指针。

参考图1了解瓦片示意图以及如何推进块指针。上述的加权求和函数如下所示：

```
import triton
import triton.language as tl

@triton.jit
def weighted_sum_fwd(
    x_ptr, weight_ptr,  # Input pointers
    output_ptr,  # Output pointer
    x_stride_row, x_stride_dim,  # Strides tell us how to move one element in each axis of a tensor
    weight_stride_dim,  # Likely 1
    output_stride_row,  # Likely 1
    ROWS, D,
    ROWS_TILE_SIZE: tl.constexpr, D_TILE_SIZE: tl.constexpr,  # Tile shapes must be known at compile time
):
    # Each instance will compute the weighted sum of a tile of rows of x.
    # `tl.program_id` gives us a way to check which thread block we're running in
    row_tile_idx = tl.program_id(0)

    # Block pointers give us a way to select from an ND region of memory
    #  and move our selection around.
    # The block pointer must know:
    # - The pointer to the first element of the tensor
    # - The overall shape of the tensor to handle out-of-bounds access
```

```
    # - The strides of each dimension to use the memory layout properly
    # - The ND coordinates of the starting block, i.e., "offsets"
    # - The block shape to use load/store at a time
    # - The order of the dimensions in memory from major to minor
    #    axes (= np.argsort(strides)) for optimizations, especially useful on H100

    x_block_ptr = tl.make_block_ptr(
        x_ptr,
        shape=(ROWS, D,),
        strides=(x_row_stride, x_stride_dim),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
        order=(1, 0),
    )

    weight_block_ptr = tl.make_block_ptr(
        weight_ptr,
        shape=(D,),
        strides=(weight_stride_dim,),
        offsets=(0,),
        block_shape=(D_TILE_SIZE,),
        order=(0,),
    )

    output_block_ptr = tl.make_block_ptr(
        output_ptr,
        shape=(ROWS,),
        strides=(output_stride_row,),
        offsets=(row_tile_idx * ROWS_TILE_SIZE,),
        block_shape=(ROWS_TILE_SIZE,),
        order=(0,),
    )

    # Initialize a buffer to write to
    output = tl.zeros((ROWS_TILE_SIZE,), dtype=tl.float32)

    for i in range(tl.cdiv(D, D_TILE_SIZE)):
        # Load the current block pointer
        # Since ROWS_TILE_SIZE might not divide ROWS, and D_TILE_SIZE might not divide D,
        # we need boundary checks for both dimensions
        row = tl.load(x_block_ptr, boundary_check=(0, 1), padding_option="zero")  # (ROWS_TILE_SIZE, D_TILE_SIZE)
        weight = tl.load(weight_block_ptr, boundary_check=(0,), padding_option="zero")  # (D_TILE_SIZE,)

        # Compute the weighted sum of the row.
        output += tl.sum(row * weight[None, :], axis=1)

        # Move the pointers to the next tile.
        # These are (rows, columns) coordinate deltas
        x_block_ptr = x_block_ptr.advance((0, D_TILE_SIZE))  # Move by D_TILE_SIZE in the last dimension
        weight_block_ptr = weight_block_ptr.advance((D_TILE_SIZE,))  # Move by D_TILE_SIZE

    # Write output to the output block pointer (a single scalar per row).
    # Since ROWS_TILE_SIZE might not divide ROWS, we need boundary checks
    tl.store(output_block_ptr, output, boundary_check=(0,))
```

现在让我们将这个内核封装成一个PyTorch Autograd函数，该函数将与PyTorch互操作（即，以Tensor作为输入，输出一个 Tensor，并在反向传播期间与autograd引擎协同工作）：

```
class WeightedSumFunc(torch.autograd.Function):
@staticmethod
    def forward(ctx, x, weight):
        # Cache x and weight to be used in the backward pass, when we
        # only receive the gradient wrt. the output tensor, and
```

```
        # need to compute the gradients wrt. x and weight.
        D, output_dims = x.shape[-1], x.shape[:-1]

        # Reshape input tensor to 2D
        input_shape = x.shape
        x = rearrange(x, "... d -> (...) d")

        ctx.save_for_backward(x, weight)

        assert len(weight.shape) == 1 and weight.shape[0] == D, "Dimension mismatch"
        assert x.is_cuda and weight.is_cuda, "Expected CUDA tensors"
        assert x.is_contiguous(), "Our pointer arithmetic will assume contiguous x"

        ctx.D_TILE_SIZE = triton.next_power_of_2(D) // 16  # Roughly 16 loops through the embedding dimension
        ctx.ROWS_TILE_SIZE = 16  # Each thread processes 16 batch elements at a time
        ctx.input_shape = input_shape

        # Need to initialize empty result tensor. Note that these elements are not necessarily 0!
        y = torch.empty(output_dims, device=x.device)

        # Launch our kernel with n instances in our 1D grid.
        n_rows = y.numel()
        weighted_sum_fwd[(cdiv(n_rows, ctx.ROWS_TILE_SIZE),)](
            x, weight,
            y,
            x.stride(0), x.stride(1),
            weight.stride(0),
            y.stride(0),
            ROWS=n_rows, D=D,
            ROWS_TILE_SIZE=ctx.ROWS_TILE_SIZE, D_TILE_SIZE=ctx.D_TILE_SIZE,
        )

        return y.view(input_shape[:-1])
```

请注意，当我们使用 `weighted_sum_fwd[(cdiv(n_rows, ctx.ROWS_TILE_SIZE),)]` 调用 Triton 内核时，通过传递元组 `(cdiv(n_rows,ctx.ROWS_TILE_SIZE),)`，我们会定义一个所谓的“启动网格”的线程块。然后，我们可以在内核中通过 `tl.program_id(0)` 访问线程块索引。

反向传播 由于我们正在定义自己的内核，我们还需要编写自己的反向函数。

在前向传播中，我们得到了我们层的输入，并需要计算其输出。在反向传播中，回想一下，我们将得到关于我们输出目标梯度的梯度，并需要计算相对于我们每个输入的梯度。在我们的情况下，我们的操作以矩阵 x： $\mathbb{R}^{n\times h}$ 和权重向量 w： $\mathbb{R}^h$ 作为输入。简而言之，让我们称我们的操作为 $f(x,w)$，其范围是 $\mathbb{R}^n$。然后，假设我们得到了 $\nabla_{f(x,w)}\mathcal{L}$，即关于我们层输出的损失 $\mathcal{L}$ 的梯度，我们可以应用多变量链式法则得到以下关于 x 和 w 的梯度表达式：

$$(\nabla_x\mathcal{L})_{ij} = \sum_{k=1}^{n} \frac{\partial f(x,w)_k}{\partial x_{ij}} (\nabla_{f(x,w)}\mathcal{L})_k = w_j \cdot (\nabla_{f(x,w)}\mathcal{L})_i \tag{2}$$

$$(\nabla_w\mathcal{L})_j = \sum_{i=1}^{n} \frac{\partial f(x,w)_i}{\partial w_j} (\nabla_{f(x,w)}\mathcal{L})_i = \sum_{i=1}^{n} x_{ij} \cdot (\nabla_{f(x,w)}\mathcal{L})_i \tag{3}$$

这给出了计算反向传递的简单公式。为了获得关于 x 的反向步，我们应用公式 2 并对 w 和 $\nabla_{f(x,w)}$ 进行外积。为了计算关于 w （即 $(\nabla_w\mathcal{L})_j$）的反向步，我们必须将我们的输入梯度乘以相应的输出行。

我们的反向传递内核将从定义所有块指针然后计算 $\nabla_x\mathcal{L}$ 开始：

```python
@triton.jit
def weighted_sum_backward(
    x_ptr, weight_ptr,  # Input
    grad_output_ptr,  # Grad input
    grad_x_ptr, partial_grad_weight_ptr,  # Grad outputs
    stride_xr, stride_xd,
    stride_wd,
    stride_gr,
    stride_gxr, stride_gxd,
    stride_gwb, stride_gwd,
    NUM_ROWS, D,
    ROWS_TILE_SIZE: tl.constexpr, D_TILE_SIZE: tl.constexpr,
):
    row_tile_idx = tl.program_id(0)
    n_row_tiles = tl.num_programs(0)

    # Inputs
    grad_output_block_ptr = tl.make_block_ptr(
        grad_output_ptr,
        shape=(NUM_ROWS,), strides=(stride_gr,),
        offsets=(row_tile_idx * ROWS_TILE_SIZE,),
        block_shape=(ROWS_TILE_SIZE,),
        order=(0,),
    )

    x_block_ptr = tl.make_block_ptr(
        x_ptr,
        shape=(NUM_ROWS, D,), strides=(stride_xr, stride_xd),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
        order=(1, 0),
    )

    weight_block_ptr = tl.make_block_ptr(
        weight_ptr,
        shape=(D,), strides=(stride_wd,),
        offsets=(0,), block_shape=(D_TILE_SIZE,),
        order=(0,),
    )

    grad_x_block_ptr = tl.make_block_ptr(
        grad_x_ptr,
        shape=(NUM_ROWS, D,), strides=(stride_gxr, stride_gxd),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
        order=(1, 0),
    )

    partial_grad_weight_block_ptr = tl.make_block_ptr(
        partial_grad_weight_ptr,
        shape=(n_row_tiles, D,), strides=(stride_gwb, stride_gwd),
        offsets=(row_tile_idx, 0),
        block_shape=(1, D_TILE_SIZE),
        order=(1, 0),
    )

    for i in range(tl.cdiv(D, D_TILE_SIZE)):
        grad_output = tl.load(grad_output_block_ptr, boundary_check=(0,), padding_option="zero")  # (ROWS_TILE_SIZE,)

        # Outer product for grad_x
        weight = tl.load(weight_block_ptr, boundary_check=(0,), padding_option="zero")  # (D_TILE_SIZE,)
        grad_x_row = grad_output[:, None] * weight[None, :]
        tl.store(grad_x_block_ptr, grad_x_row, boundary_check=(0, 1))

        # Reduce as many rows as possible for the grad_weight result
```

```
        row = tl.load(x_block_ptr, boundary_check=(0, 1), padding_option="zero")  # (ROWS_TILE_SIZE, D_TILE_SIZE)
        grad_weight_row = tl.sum(row * grad_output[:, None], axis=0, keep_dims=True)
        tl.store(partial_grad_weight_block_ptr, grad_weight_row, boundary_check=(1,))  # Never out of bounds for dim 0

        # Move the pointers to the next tile along D
        x_block_ptr = x_block_ptr.advance((0, D_TILE_SIZE))
        weight_block_ptr = weight_block_ptr.advance((D_TILE_SIZE,))
        partial_grad_weight_block_ptr = partial_grad_weight_block_ptr.advance((0, D_TILE_SIZE))
        grad_x_block_ptr = grad_x_block_ptr.advance((0, D_TILE_SIZE))
```

计算梯度 ∇_x 很简单，我们将结果写入输出张量的相应瓦片。然而，计算 ∇_w 稍微更具挑战性。每个内核实例负责 x 的一行瓦片，但我们现在需要跨 x 的行进行求和。我们不会在反向传递中直接执行此求和，而是假设 `partial_grad_weight_ptr` 包含一个 `n_row_tiles` $\times H$ 矩阵，其中第一个维度仅在 x 的行瓦片内进行缩减。我们在写入此张量之前，会在当前行瓦片内进行缩减。在内核外部，我们使用 `torch.sum` 来缩减 ∇_w，以汇总来自每个行瓦片[1]的结果。然后，`autograd.Function` 的最后部分就相对简单了：

```
class WeightedSumFunc(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, weight):
        # ... (defined earlier)

    @staticmethod
    def backward(ctx, grad_out):
        x, weight = ctx.saved_tensors
        ROWS_TILE_SIZE, D_TILE_SIZE = ctx.ROWS_TILE_SIZE, ctx.D_TILE_SIZE  # These don't have to be the same
        n_rows, D = x.shape

        # Our strategy is for each thread block to first write to a partial buffer,
        # then we reduce over this buffer to get the final gradient.
        partial_grad_weight = torch.empty((cdiv(n_rows, ROWS_TILE_SIZE), D), device=x.device, dtype=x.dtype)
        grad_x = torch.empty_like(x)

        weighted_sum_backward[(cdiv(n_rows, ROWS_TILE_SIZE),)](
            x, weight,
            grad_out,
            grad_x, partial_grad_weight,
            x.stride(0), x.stride(1),
            weight.stride(0),
            grad_out.stride(0),
            grad_x.stride(0), grad_x.stride(1),
            partial_grad_weight.stride(0), partial_grad_weight.stride(1),
            NUM_ROWS=n_rows, D=D,
            ROWS_TILE_SIZE=ROWS_TILE_SIZE, D_TILE_SIZE=D_TILE_SIZE,
        )
        grad_weight = partial_grad_weight.sum(axis=0)
        return grad_x, grad_weight
```

最后，我们现在可以获得一个与 `torch.nn.functional` 中实现的那些功能非常相似的函数：

```
f_weightedsum = WeightedSumFunc.apply
```

现在，调用 `f_weightedsum` 在两个 PyTorch 张量 x 和 w 上将给出如下所示的张量：

```
tensor([ 90.8563, -93.6815, -80.8884,  ..., 103.4840, -21.4634, -24.0192],
       device='cuda:0', grad_fn=<WeightedSumFuncBackward>)
```

注意与张量关联的 `grad_fn` — 这表明 PyTorch 在反向传播过程中知道在计算图中出现此张量时应该调用什么。这完成了我们对加权求和操作的 Triton 实现。

[1]当然，我们也可以为此编写自己的内核。

1.3.2 FlashAttention-2 前向传递

您将用基于 FlashAttention-2 [Dao，2023] 的显著改进的 Triton 实现来替换您的 PyTorch 注意力实现。FlashAttention-2 采用了一些技巧来以瓦片为单位计算前向传递，这允许高效的内存访问模式，并避免了在全局内存上实现完整注意力矩阵的需要。

在进入本节之前，我们强烈建议至少阅读一下原始的 FlashAttention 论文[Dao 等人， 2022], ，这将为您理解 FlashAttention 实现高效注意力计算的核心技术——在瓦片（ [Milakov 和 Gimelshein 提出的技术 2018]）上以在线方式计算 softmax——提供直觉。我们还建议阅读 He [2022] ，以了解 GPU 实际执行 PyTorch 代码的方式。

理解原版注意力机制中的低效性。回想一下，忽略掩码的情况下，注意力的前向传播可以写成：

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top/\sqrt{d} \tag{4}$$
$$\mathbf{P}_{ij} = \mathrm{softmax}_j(\mathbf{S})_{ij} \tag{5}$$
$$\mathbf{O} = \mathbf{P}\mathbf{V} \tag{6}$$

标准反向传播是

$$\mathbf{dV} = \mathbf{P}^\top\mathbf{dO} \tag{7}$$
$$\mathbf{dP} = \mathbf{dO}\mathbf{V}^\top \tag{8}$$
$$\mathbf{dS}_i = \mathrm{dsoftmax}(\mathbf{dP}_i) = \left(\mathrm{diag}(\mathbf{P}_i) - \mathbf{P}_i\mathbf{P}_i^\top\right)\mathbf{dP}_i \tag{9}$$
$$\mathbf{dQ} = \mathbf{dS}\mathbf{K}/\sqrt{d} \tag{10}$$
$$\mathbf{dK} = \mathbf{dS}^\top\mathbf{Q}/\sqrt{d}, \tag{11}$$

如我们所见，反向传播依赖于前向传播中的一些非常大的激活值。例如，计算公式(7)中的dV需要P，P是形状为 (`batch_size`，`n_heads`，`seq_len`，`seq_len`)的注意力分数——这个激活矩阵的大小与序列长度呈平方关系，解释了我们在大序列长度下基准测试注意力时遇到的上文提到的内存问题。在香草注意力的前向和反向传播过程中，我们将P和其他大型激活值在片上SRAM和GPU HBM之间传输，支付了显著的内存IO成本。标准实现中有几次这样的传输：例如，标准的反向传播实现会在计算公式(7)和(9)时从HBM读取P。

FlashAttention 的主要目标是避免将注意力矩阵读入和写出到 HBM，以减少 IO 和峰值内存成本。我们通过三种技术实现这一点：tiling、recomputation 和 operator fusion。

Tiling。为了避免将注意力矩阵读入和写出到 HBM，我们在无需访问整个输入的情况下计算 softmax 还原。具体来说，我们重构注意力计算，将输入拆分为多个 tile，并对输入 tile 进行多次遍历，从而逐步执行 softmax 还原。

重新计算。我们避免在 HBM 中存储形状为 (`batch_size`，`n_heads`，`seq_len`，`seq_len`) 的大型中间注意力矩阵。相反，我们将会在 HBM 中保存某些“激活检查点”，然后在反向传递期间重新计算前向传递的一部分，以获取我们用于计算梯度的其他所需激活。FlashAttention-2 还存储注意力分数的 logsumexp， L，这将

用于简化反向传播计算。 L 的表达式为:

$$L_i = \log\left(\sum_j \exp\left(\mathbf{S}_{ij}\right)\right) \tag{12}$$

在我们的最终内核中，我们将以在线方式计算这一点，但最终结果应该是相同的。通过结合分块和重新计算，我们的内存IO和峰值使用不再依赖于 `sequence_length`^{2}，因此我们可以使用更长的序列长度。

算子融合。最后，我们通过在一个内核中执行所有操作来避免注意力矩阵和其他中间激活的重复内存IO——这被称为算子或内核融合。我们将为前向传播编写一个Triton内核，该内核执行注意力中涉及的所有操作，并在HBM和SRAM之间进行有限的数据传输。算子融合部分得益于重新计算，因为我们避免了通常需要支付以将每个中间激活存储到HBM的内存IO。

要了解更多关于这些技术的直观理解，请查阅FlashAttention论文 [Dao等人，2022年，Dao， 2023]。

使用重新计算的反向传播。通过 L，我们可以执行适当的重新计算并高效地计算反向传播。在开始反向传播之前，我们将值 $D = \mathrm{rowsum}(\mathrm{O} \circ \mathrm{dO})$（其中 $\circ$ 是逐元素乘法）预计算到全局内存中，该值等于 $\mathrm{rowsum}(\mathrm{P} \circ \mathrm{dP})$，因为 $\mathbf{PdP}^\top = \mathbf{P}(\mathbf{dOV}^\top)^\top = (\mathbf{PV})\mathbf{dO}^\top = \mathbf{OdO}^\top$ （以及对于任何矩阵A和B的rowsum $(\mathbf{A} \circ \mathbf{B}) = \mathrm{diag}(\mathbf{AB}^\top)$ ）。有了 L 和 D 向量，反向传播可以在没有softmax的情况下计算。反向传播的完整计算现在如下:

$$\mathbf{S} = \mathbf{QK}^\top / \sqrt{d} \tag{13}$$
$$\mathbf{P}_{ij} = \exp\left(\mathbf{S}_{ij} - L_i\right) \tag{14}$$
$$\mathbf{dV} = \mathbf{P}^\top \mathbf{dO} \tag{15}$$
$$\mathbf{dP} = \mathbf{dOV}^\top \tag{16}$$
$$\mathbf{dS}_{ij} = \mathbf{P}_{ij} \circ (\mathbf{dP}_{ij} - D_i) \tag{17}$$
$$\mathbf{dQ} = \mathbf{dSK} / \sqrt{d} \tag{18}$$
$$\mathbf{dK} = \mathbf{dS}^\top \mathbf{Q} / \sqrt{d}, \tag{19}$$

我们可以看到，操作序列的执行并不需要我们在前向传递过程中将注意力分数 P 存储在 HBM 中——我们从激活值 Q、K 和 (13) 及 (14) 中的 L 重新计算它们。

闪存注意力前向传递的细节。现在我们已经对 FlashAttention-2 中使用的技巧有了高层级的了解，我们将深入探讨您将要实现的 FA2 前向传递内核的细节。为了避免将注意力矩阵读入和写出到 HBM，我们希望使用分块处理，即独立于其他块地计算每个输出块。这要求我们能够计算 P 的块，理想情况下在两个维度（用于查询和用于键）上都是分块的。

然而，当我们对 S 应用 softmax 时，我们需要将 S 的整行进行归约以计算 softmax 分母，这意味着我们不能直接分块计算 P 。FlashAttention-2 通过在线 softmax 解决了这个问题。在接下来的文本中，我们将使用下标索引 i 来表示当前的查询块，使用上标索引 (j) 来表示当前键块。查询维度上的块大小为 B_q，键维度为 B_k。我们将不在隐藏维度 d 上进行分块。

我们还保留了一些按行计算的运行值， $m_i^{(j)} \in \mathbb{R}^{B_q}$ 和 $l_i^{(j)} \in \mathbb{R}^{B_q}$。按行的 $m_i^{(j)}$ 值是一个运行最大值，我们跟踪它，以便以数值稳定的方式计算softmax（回想一下我们在作业1中softmax实现中的这个技巧）。我们将用每个新的按行 S 瓦片更新 $m_i^{(j)}$ （当 j 增加时）。使用运行最大值，我们可以计算未归一化的softmax值

（分子）作为 $\tilde{\mathbf{P}}_i^{(j)} = \exp\left(\mathbf{S}_{ij} - m_i^{(j)}\right)$。 $l_i^{(j)}$ 是softmax分母的运行代理，将使用未归一化的softmax值更新为 $l_i^{(j)} = \exp(m_i^{(j-1)})$。当我们最终写入输出时，我们需要使用 $l_i^{(T_k)}$ 完成归一化， $l_i^{(T_k)}$ 是处理所有键瓦片后的 $l_i^{(j)}$ 的最终值。算法1显示了应该在GPU上实现的正向传递

Algorithm 1 FlashAttention-2 forward pass

Require: $\mathbf{Q} \in \mathbb{R}^{N_q \times d}$, $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{N_k \times d}$, tile sizes B_q, B_k

Split $\mathbf{Q}$ into $T_q = \left\lceil \frac{N_q}{B_q} \right\rceil$ tiles $\mathbf{Q}_1, \ldots, \mathbf{Q}_{T_q}$ of size $B_q \times d$

Split $\mathbf{K}, \mathbf{V}$ into $T_k = \left\lceil \frac{N_k}{B_k} \right\rceil$ tiles $\mathbf{K}^{(1)}, \ldots, \mathbf{K}^{(T_k)}$ and $\mathbf{V}^{(1)}, \ldots, \mathbf{V}^{(T_k)}$ of size $B_k \times d$

for $i = 1, \ldots, T_q$ **do**

  Load $\mathbf{Q}_i$ from global memory

  Initialize $\mathbf{O}_i^{(0)} = \mathbf{0} \in \mathbb{R}^{B_q \times d}, l_i^{(0)} = 0 \in \mathbb{R}^{B_q}, m_i^{(0)} = -\infty \in \mathbb{R}^{B_q}$

  for $j = 1, \ldots, T_k$ **do**

    Load $\mathbf{K}^{(j)}, \mathbf{V}^{(j)}$ from global memory

    Compute tile of pre-softmax attention scores $\mathbf{S}_i^{(j)} = \frac{\mathbf{Q}_i (\mathbf{K}^{(j)})^\top}{\sqrt{d}} \in \mathbb{R}^{B_q \times B_k}$

    Compute $m_i^{(j)} = \max\left(m_i^{(j-1)}, \text{rowmax}\left(\mathbf{S}_i^{(j)}\right)\right) \in \mathbb{R}^{B_q}$

    Compute $\tilde{\mathbf{P}}_i^{(j)} = \exp\left(\mathbf{S}_i^{(j)} - m_i^{(j)}\right) \in \mathbb{R}^{B_q \times B_k}$

    Compute $l_i^{(j)} = \exp\left(m_i^{(j-1)} - m_i^{(j)}\right) l_i^{(j-1)} + \text{rowsum}\left(\tilde{\mathbf{P}}_i^{(j)}\right) \in \mathbb{R}^{B_q}$

    Compute $\mathbf{O}_i^{(j)} = \text{diag}\left(\exp\left(m_i^{(j-1)} - m_i^{(j)}\right)\right) \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}^{(j)}$

  end for

  Compute $\mathbf{O}_i = \text{diag}\left(l_i^{(T_k)}\right)^{-1} \mathbf{O}_i^{(T_k)}$

  Compute $L_i = m_i^{(T_k)} + \log\left(l_i^{(T_k)}\right)$

  Write $\mathbf{O}_i$ to global memory as the i-th tile of $\mathbf{O}$.

  Write L_i to global memory as the i-th tile of L.

end for

Return the output $\mathbf{O}$ and the logsumexp L.

在开始实现 Triton 中的前向传递之前，我们先在此收集一些关于编写 Triton 内核的通用技巧和窍门。

Triton 技巧与窍门

- 您可以在 Triton 中使用 `tl.device_print` print 语句进行调试： `https://triton-lang.org/main/python-api/generated/triton.language.device_print.html`。有一个设置 `TRITON_INTERPRET=1` 可以在 CPU 上运行 Triton 解释器，尽管我们发现它有错误。
- 在定义块指针时，请确保它们具有正确的偏移量，并且块偏移量应乘以适当的瓦片大小。
- 线程块的启动网格是使用

  ```
  kernel_fn[(launch_grid_d1, launch_grid_d2, ...)](...arguments...)
  ```

  在 `torch.autograd.Function` 子类的 `torch.autograd.Function` 方法中，正如我们在加权求和示例中所见的那样。

- 使用 `tl.dot` 执行矩阵乘法。
- 要推进块指针，请使用 `*_block_ptr = *_block_ptr.advance(...)`

问题 (flash_forward)：15 分

(a) 编写一个纯 PyTorch（不使用 Triton）的 `autograd.Function`，实现 FlashAttention-2 的正向传递。这会比常规的 PyTorch 实现慢很多，但将有助于你调试你的 Triton 内核。

您的实现应接受输入Q、K和V，以及一个标志 `is_causal`，并产生输出O和logsumexp值 L。对于此任务，您可以忽略 `is_causal` 标志。然后`autograd.Function`前向应使用save L, Q, K, V, O 为反向传播，并返回 O。请记住，`forward` 方法的 `autograd.Function`实现始终将上下文作为其第一个参数。任何 `autograd.Function` 类都需要实现一个 `backward` 方法，但目前您可以让它仅引发 `NotImplementedError`。如果您需要比较的对象，您可以实现PyTorch中的方程式4到6和12，并比较您的输出。

接口随后是 `def forward(ctx, Q, K, V, is_causal=False)`。确定你自己的瓦片大小，但确保它们至少为 16×16 大小。我们将始终使用至少为 16 的 2 的幂次方尺寸来测试你的代码，因此你不需要担心越界访问。

交付物：一个 `torch.autograd.Function`，在正向传递中实现 FlashAttention-2 的子类。为了测试你的代码，请实现 `[adapters.get_flashattention_autograd_function_pytorch]`。然后，使用 `uv run pytest -k test_flash_forward_pass_pytorch` 运行测试，并确保您的实现通过它。

(b) 根据 Algorithm 1 为 FlashAttention-2 的前向传递编写一个 Triton 内核。然后，编写另一个 `torch.autograd.Function` 子类，它在前向传递中调用这个（融合的）内核，而不是在 PyTorch 中计算结果。一些特定问题的提示：

- 为了调试，我们建议将您执行的每个 Triton 操作的结果与在 (a) 部分编写的分块 PyTorch 实现进行比较。
- 您的启动网格应设置为 $(T_q, \texttt{batch_size})$，这意味着每个 Triton 程序实例将仅加载来自单个 batchindex 的元素，并且仅读取/写入 Q、O 和 L 的单个查询瓦片。
- 内核应只有一个循环，该循环将迭代键瓦片 $1 \leq j \leq T_k$。
- 在循环末尾推进块指针。
- 使用下方函数声明（使用我们提供的块指针，你应该能够推断其余指针的设置）：

```
@triton.jit
def flash_fwd_kernel(
    Q_ptr, K_ptr, V_ptr,
    O_ptr, L_ptr,
    stride_qb, stride_qq, stride_qd,
    stride_kb, stride_kk, stride_kd,
    stride_vb, stride_vk, stride_vd,
    stride_ob, stride_oq, stride_od,
    stride_lb, stride_lq
```

```
    N_QUERIES, N_KEYS,
    scale,
    D: tl.constexpr,
    Q_TILE_SIZE: tl.constexpr,
    K_TILE_SIZE: tl.constexpr,
):
    # Program indices
    query_tile_index = tl.program_id(0)
    batch_index = tl.program_id(1)

    # Offset each pointer with the corresponding batch index
    # multiplied with the batch stride for each tensor
    Q_block_ptr = tl.make_block_ptr(
        Q_ptr + batch_index * stride_qb,
        shape=(N_QUERIES, D),
        strides=(stride_qq, stride_qd),
        offsets=(query_tile_index * Q_TILE_SIZE, 0),
        block_shape=(Q_TILE_SIZE, D),
        order=(1, 0),
    )

    ...
```

在哪里 `scale` 是 $\frac{1}{\sqrt{d}}$ 并且 `Q_TILE_SIZE` 并且 `K_TILE_SIZE` 分别是 B_q 并且 B_k 。你可以稍后调整这些。

这些附加指南可能有助于您避免精度问题：

- 片上缓冲区（O_i, l, m）应具有 `dtype` `tl.float32`。如果您正在累积到输出缓冲区，请使用 `acc` 参数 (`acc = tl.dot(..., acc=acc)`)。
- 在将它们相乘之前，将 $\tilde{\mathrm{P}}(j)_i$ 转换为 $\mathrm{V}(j)$ 的 `dtype` ，并在将 O_i 写入全局内存之前将其转换为适当的 `dtype` 。转换使用 `tensor.to` 完成。您可以使用 `tensor.dtype` 获取张量的 `dtype` ，使用 `*_block_ptr.type.element_ty` 获取块指针 / 指针的 dtype。

交付物：一个 `torch.autograd.Function` 子类，在正向传递中实现 FlashAttention-2，使用您的 Triton 内核。实现 `torch.autograd.Function` 。然后，使用 运行测试

并确保您的实现通过它。

`-k test_flash_forward_pass_triton` 并确保您的实现通过它。

(c) 在您的 `autograd.Function` 实现中作为最后一个参数添加一个标志以用于因果掩码。这应该是一个布尔标志，当设置为 `True`时启用因果掩码的索引比较。您的 Triton 内核应该有一个相应的附加参数`is_causal: tl.constexpr` （这是一个必需的类型注解）。在 Triton 中，为查询和键构建适当的索引向量，并将它们进行比较以形成一个大小为$B_q \times B_k$的正方形掩码。对于被掩码的元素，将常数值 `-1e6` 添加到注意力分数矩阵 $\mathrm{S}_i^{(j)}$ 的相应元素中。确保使用 `ctx.is_causal = is_causal` 保存掩码标志以用于反向传播。

可交付成果：为您的`torch.autograd.Function` 子类实现一个额外的标志
使用您的 Triton 内核实现 FlashAttention-2 前向传递和因果掩码的 `False` 子类。确保标志是可选的，默认值为 `False` ，以便之前的测试仍然通过。

实现反向传播的重新计算 注意到与公式7到11中的标准反向传播不同，我们可以使用重新计算来避免公式13到19中反向传播中的softmax操作。这意味着我们可以使用简单的内核来计算反向传播，并且不需要在线技巧。因此，对于这部分，你可以通过在常规的 `torch.compile` 上调用 `torch.compile` 来实现反向传播

PyTorch 函数（非 Triton）。

问题 (`flash_backward`)：5 分

使用PyTorch（不是Triton）和 `torch.compile`实现你的FlashAttention-2 `autograd.Function` 的反向传播。你的实现应该以Q、K、V、O、dO和 L 张量作为输出，并返回dQ、dK和dV。记住要计算和使用 D 向量。你可以参考公式13到19的计算过程。

可交付成果： 要测试您的实现，请运行 `uv run pytest -k test_flash_backward`。

现在，让我们比较您的（部分）Triton FlashAttention-2 实现与您的 PyTorch 常规 Attention 实现的性能。

问题 (`flash_benchmarking`)：5 分

(a) 使用 `triton.testing.do_bench` 编写一个基准测试脚本，比较你的（部分）Triton FlashAttention-2 前向和后向传递的实现与常规 PyTorch 实现的性能（即不使用 FlashAttention）。

具体来说，您将报告一个包含正向、反向以及结束延迟的表格
向前-向后端到端传递，对你的Triton和PyTorch实现。在开始基准测试之前，随机生成任何必要的输入，并在单个H100上运行基准测试。始终使用批大小1和因果掩码。扫描2的幂次方序列长度从128到65536的笛卡尔积，嵌入维度大小从16到128的2的幂次方，以及精度为 `torch.bfloat16` 和 `torch.float32`。你将
可能需要根据输入尺寸调整瓦片大小。

可交付成果：一个比较您的 FlashAttention-2 实现与 PyTorch 实现的表格，使用上述设置，并报告前向、后向和端到端的
延迟结束。

1.3.3 FlashAttention-2 排行榜

作业 2 的排行榜将测试你实现 FlashAttention-2 的速度（包括前向和后向传递）。我们挑战你进一步优化你的实现性能，使用你能想到的任何技巧。限制条件是你不能改变函数的输入/输出，并且必须使用 Triton（不幸的是不能使用 CUDA）。你的输入将在 BF16 模式下进行因果掩码测试，并且必须通过与你常规实现相同的测试。该实现也必须是你的原创，不能使用现有的实现。你的计时应在 H100 上进行，样本批大小为 1，查询、键和值的序列长度为 16,384，以及 $d_{\text{model}} = 102416$ 个头。我们将验证前 5-10 个提交的正确性和性能。我们将运行以下测试来计时你的实现：

```python
def test_timing_flash_forward_backward():
    n_heads = 16
    d_head = 64
    sequence_length = 16384
    q, k, v = torch.randn(
        3, n_heads, sequence_length, d_head, device='cuda', dtype=torch.bfloat16, requires_grad=True
    )

    flash = torch.compile(FlashAttention2.apply)
```

```python

def flash_forward_backward():
    o = flash(q, k, v, True)
    loss = o.sum()
    loss.backward()

results = triton.testing.do_bench(flash_forward_backward, rep=10000, warmup=1000)
print(results)
```

为测试目的，你可以将重复和预热时间（以毫秒为单位）减少到更短的时间。一些改进建议：

- 调整你的内核的瓦片大小（使用Triton自动调优！）
- 调整额外的Triton配置参数
- 在Triton中实现反向传播，而不仅仅是 `torch.compile` （见下文1.3.4节）
- 对输入进行两次反向传播，一次用于dQ，另一次用于dK和dV，以避免原子操作或块之间的同步。

- 在进行因果掩码时提前停止程序实例，跳过所有始终为零的瓦片
- 将未遮盖的瓦片与瓦片对角线分离，计算前者时永不比较索引，计算后者时仅进行一次比较indices，和后者仅进行一次比较
- 在H100上使用TMA（张量内存加速器）功能，遵循与此教程类似的模式

将你的最佳成绩提交到排行榜上

`github.com/stanford-cs336/assignment2-systems-leaderboard`

1.3.4 可选：Triton反向传播

如果你对使用Triton进行更多练习感兴趣，或者想快速提交排行榜成绩，我们提供了以下分块FlashAttention-2反向传播，你可以在Triton中实现它。算法2展示了应该如何在Triton中实现FlashAttention-2反向传播。这里的技巧在于将P计算两次，一次用于dQ的反向传播，另一次用于dK和dV。这让我们可以跳过线程块之间的同步。

Algorithm 2 Tiled FlashAttention-2 backward pass

Require: $\mathbf{Q}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N_q \times d}$, $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{N_k \times d}$, $L \in \mathbb{R}^{N_q}$, tile sizes B_q, B_k

Compute $D = \mathrm{rowsum}(\mathbf{dO} \circ \mathbf{O}) \in \mathbb{R}^{N_q}$

Split $\mathbf{Q}, \mathbf{O}, \mathbf{dO}$ into $T_q = \left\lceil \frac{N_q}{B_q} \right\rceil$ tiles $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_q}$, $\mathbf{O}_1, \dots, \mathbf{O}_{T_q}$, $\mathbf{dO}_1, \dots, \mathbf{dO}_{T_q}$, each of size $B_q \times d$

Split $\mathbf{K}, \mathbf{V}$ into $T_k = \left\lceil \frac{N_k}{B_k} \right\rceil$ tiles $\mathbf{K}^{(1)}, \dots, \mathbf{K}^{(T_k)}$ and $\mathbf{V}^{(1)}, \dots, \mathbf{V}^{(T_k)}$, each of size $B_k \times d$

Split L, D into T_q tiles $L_1, \dots, L_{T_q}$ and $D_1, \dots, D_{T_q}$, each of size B_q

for $j = 1, \dots, T_k$ **do**

  Load $\mathbf{K}^{(j)}, \mathbf{V}^{(j)}$ from global memory

  Initialize $\mathbf{dK}^{(j)} = \mathbf{dV}^{(j)} = \mathbf{0} \in \mathbb{R}^{B_k \times d}$

  for $i = 1, \dots, T_q$ **do**

    Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i$ from global memory

    Compute tile of attention scores $\mathbf{S}_i^{(j)} = \frac{\mathbf{Q}_i (\mathbf{K}^{(j)})^\top}{\sqrt{d}} \in R^{B_q \times B_k}$

    Compute attention probabilities $\mathbf{P}_i^{(j)} = \exp\left(\mathbf{S}_i^{(j)} - L_i\right) \in \mathbb{R}^{B_q \times B_k}$

    Compute $\mathbf{dV}^{(j)} \mathrel{+}= (\mathbf{P}_i^{(j)})^\top \mathbf{dO}_i \in \mathbb{R}^{B_k \times d}$

    Compute $\mathbf{dP}_i^{(j)} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_q \times B_k}$

    Compute $\mathbf{dS}_i^{(j)} = \mathbf{P}_i^{(j)} \circ \left(\mathbf{dP}_i^{(j)} - D_i\right) / \sqrt{d} \in \mathbb{R}^{B_q \times B_k}$

    Load $\mathbf{dQ}_i$ from global memory, then update $\mathbf{dQ}_i \mathrel{+}= \mathbf{dS}_i^{(j)} \mathbf{K}^{(j)} \in \mathbb{R}^{B_q \times d}$, and write back to global memory. Must be atomic for correctness!

    Compute $\mathbf{dK}^{(j)} \mathrel{+}= (\mathbf{dS}_i^{(j)})^\top \mathbf{Q}_i \in \mathbb{R}^{B_k \times d}$.

  end for

  Write $\mathbf{dK}^{(j)}$ and $\mathbf{dV}^{(j)}$ to global memory as the j-th tiles of $\mathbf{dK}$ and $\mathbf{dV}$.

end for

Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

2 分布式数据并行训练

在作业的下一部分，我们将探讨使用多个GPU训练语言模型的方法，重点关注数据并行性。我们将从PyTorch中的分布式通信基础开始。然后，我们将研究分布式数据并行训练的朴素实现，接着实现和基准测试各种改进以提高通信效率。

2.1 单节点分布式通信（PyTorch）

让我们从PyTorch中的一个简单分布式应用程序开始，目标是生成四个随机整数张量并计算它们的和。

在下面的分布式情况下，我们将启动四个工作进程，每个进程生成一个随机整数张量。为了在工作进程之间求和这些张量，我们将调用all-reduce集体通信操作，该操作用每个进程上的所有约简结果（即和）替换原始数据张量。

现在让我们看看一些代码。

```python
import os
import torch
import torch.distributed as dist
import torch.multiprocessing as mp

def setup(rank, world_size):
    os.environ["MASTER_ADDR"] = "localhost"
    os.environ["MASTER_PORT"] = "29500"
    dist.init_process_group("gloo", rank=rank, world_size=world_size)

def distributed_demo(rank, world_size):
    setup(rank, world_size)
    data = torch.randint(0, 10, (3,))
    print(f"rank {rank} data (before all-reduce): {data}")
    dist.all_reduce(data, async_op=False)
    print(f"rank {rank} data (after all-reduce): {data}")

if __name__ == "__main__":
    world_size = 4
    mp.spawn(fn=distributed_demo, args=(world_size, ), nprocs=world_size, join=True)
```

运行上述脚本后，我们得到如下输出。正如预期，每个工作进程最初持有不同的 `data` 张量。在跨所有工作进程对张量进行求和的全归约操作后， `data` 在每个工作进程上原地被修改以持有全归约结果.[2]

```
$ uv run python distributed_hello_world.py
rank 3 data (before all-reduce): tensor([3, 7, 8])
rank 0 data (before all-reduce): tensor([4, 4, 7])
rank 2 data (before all-reduce): tensor([6, 0, 7])
rank 1 data (before all-reduce): tensor([9, 5, 3])
rank 1 data (after all-reduce): tensor([22, 16, 25])
rank 0 data (after all-reduce): tensor([22, 16, 25])
```

[2]如果你多次运行这个脚本，你会发现打印输出的顺序是不确定的。由于这个应用程序是在分布式环境中运行的，我们无法控制命令被执行的精确顺序——我们唯一能保证的是，在 all-reduce 操作完成后，各个进程将持有位级完全相同的张量结果。

```
rank 3 data (after all-reduce): tensor([22, 16, 25])
rank 2 data (after all-reduce): tensor([22, 16, 25])
```

现在让我们更仔细地回顾一下上面的脚本。命令 `mp.spawn` 会生成 `nprocs` 个进程，这些进程以提供的 `args` 运行。此外，函数 `fn` 会被作为 `fn(rank, *args)` 调用，其中 rank 是工作进程的索引（一个介于 0 和 `nprocs-1` 之间的值）。因此，我们的 `distributed_demo` 函数必须将其作为第一个位置参数接受这个整数 rank。此外，我们还传递了 `world_size`，它指的是工作进程的总数。

这些工作进程都属于一个进程组，该进程组通过 `dist.init_process_group` 初始化。进程组代表多个将通过网络主节点进行协调和通信的工作进程。主节点由其 IP 地址和端口定义，并且主节点运行 rank 为 0 的进程。集体通信操作（如 all-reduce）会在进程组中的每个进程上执行。

在这种情况下，我们使用 `"gloo"` 后端初始化了我们的进程组，但还有其他后端可用。特别是，`"nccl"` 后端将使用 NVIDIA NCCL 集体通信库，这通常对 CUDA 张量更高效。然而，NCCL 只能在配备 GPU 的机器上使用，而 Gloo 可以在仅 CPU 的机器上运行。一个有用的经验法则是：对于分布式 GPU 训练使用 NCCL，对于分布式 CPU 训练和/或本地开发使用 Gloo。在这个例子中，我们使用了 Gloo，因为它允许在仅 CPU 的机器上进行本地执行和开发。

在运行多GPU任务时，确保不同rank使用不同的GPU。一种方法是在设置函数中调用 `torch.cuda.set_device(rank)`），以便 `tensor.to("cuda")`会自动将其移动到指定的设备。或者，您可以显式创建每个rank的设备字符串（例如，`device = f"cuda:{rank}"`），然后使用此设备字符串作为任何数据移动（例如，`tensor.to(f"cuda:{rank}")`）的目标设备。

术语。在作业的其余部分（以及您在线上可能看到的各种其他资源）中，您可能会在PyTorch分布式通信的上下文中遇到以下术语。尽管在本作业中我们将专注于单节点、多进程分布式训练，但这些术语对于理解分布式训练的总体情况很有用。参见图2以获取视觉表示。

节点：网络上的机器。

工作进程：参与分布式训练的程序实例。在本作业中，每个工作进程将只有一个进程，因此我们将交替使用工作进程、进程和工作进程。然而，工作进程可能使用多个进程（例如，用于加载数据进行训练），因此这些术语在实践中并不总是等效的。

世界规模：一个进程组中的总工作线程数。

全局排名：一个整数ID（介于0和 `world_size-1`之间），用于唯一标识进程组中的一个工作线程。例如，对于规模为2的世界，一个进程将具有全局排名0（主进程），另一个进程将具有排名1。

本地世界规模：当跨不同节点运行应用程序时，本地世界规模是指在一个给定节点上本地运行的工作线程数。例如，如果我们有一个应用程序，在2个节点上每个节点生成4个工作线程，那么世界规模将是8，而本地世界规模将是4。注意，在单个节点上运行时，工作线程的本地世界规模等同于（全局）世界规模。

local rank: 一个整数 ID（介于 0 和 `local_world_size-1` 之间），用于唯一标识机器上本地工作者的索引。例如，如果我们有一个在每 2 个节点上生成 4 个进程的应用程序，那么每个节点上的工作者将具有本地 rank 0、1、2 和 3。请注意，在运行单节点多进程分布式应用程序时，进程的本地 rank 等同于其全局 rank。

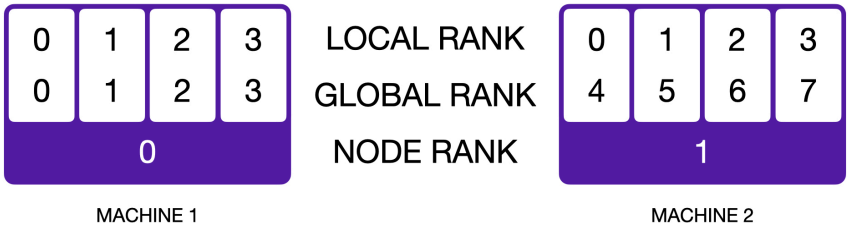

图2：一个在2个节点上运行、世界大小为8的分布式应用的示意图。每个工作进程由一个全局排名（从0到7）和一个本地排名（从0到3）标识。图取自lightning.ai/docs/fabric/stable/advanced/distributed_communication.html

2.1.1 分布式应用基准测试的最佳实践

在整个作业的这个部分中，你将基准测试分布式应用程序，以更好地理解通信带来的开销。这里有一些最佳实践：

- 尽可能在相同机器上运行基准测试，以便进行受控比较。
- 在计时感兴趣的操作之前执行几个预热步骤。这对于NCCL通信调用尤其重要。通常预热5次就足够了。
- 在GPU上进行基准测试时，调用 `torch.cuda.synchronize()` 来等待CUDA操作完成。请注意，即使调用使用 `async_op=False`的通信操作（该操作在GPU上排队时返回，而不是通信实际完成时返回），这也必不可少。[3]
- 不同排名的计时可能会有细微差异，因此通常需要跨排名聚合测量值以改进估计。您可能会发现全收集集体（特别是 `dist.all_gather_object` 函数）对于收集所有排名的结果很有用。
- 通常，在本地使用 CPU 上的 Gloo 进行调试，然后在特定问题的需要下，使用 NCCL 在 GPU 上进行基准测试。切换后端只需更改 `init_process_group` 调用和张量设备转换。

问题 (`distributed_communication_single_node`)：5 分

编写一个脚本来基准测试单节点多进程设置中全归约操作的运行时。上面的示例代码可能是一个合理的起点。尝试更改以下设置：

后端 ＋ 设备类型：Gloo +CPU，NCCL +GPU。

全归约数据大小：范围在 1MB、10MB、100MB、1GB 的 float32 数据张量。

进程数量：2、4 或 6 个进程。

资源需求：最多6个GPU。每次基准测试应在5分钟以内完成。

[3]请查看 github.com/pytorch/pytorch/issues/68112#issuecomment-965932386 获取更多详细信息。

可交付成果：比较不同设置的情节（s）和/或表格（s），并附有 2-3 句关于您结果和想法的评论，说明各种因素如何相互作用。

2.2 一种简单的分布式数据并行训练实现

既然我们已经了解了PyTorch中编写分布式应用的基础，那么让我们构建一个分布式数据并行（DDP）训练的最小实现。

数据并行将批次拆分到多个设备（例如 GPU）上，从而能够在单个设备上无法容纳的大批量上进行训练。例如，如果有四个每个最多可以处理 32 个批量大小的设备，数据并行训练将使有效批量大小达到 128。

以下是进行简单分布式数据并行训练的步骤。最初，每个设备构建一个（随机初始化的）模型。我们使用广播集体通信操作将模型参数从 rank 0 发送到所有其他 rank。在训练开始时，每个设备都持有相同的模型参数和优化器状态（例如 Adam 中的累积梯度统计信息）。

1. 给定一个包含 n 个示例的批次，该批次会被分片，每个设备接收 n/d 个不重叠的示例（其中 d 是用于数据并行训练的设备数量）。 n 应该能整除 d，因为训练时间受最慢进程的限制。

2. 每个设备使用其本地模型参数在 n/d 示例上运行前向传递，并在反向传递中计算梯度。请注意，此时每个设备都持有从其接收到的 n/d 示例计算出的梯度。

3. 我们然后使用全归约（all-reduce）集体通信操作来跨不同设备平均梯度。
4. 这样，每个设备都持有跨所有 n 示例平均后的梯度。

5. 接下来，每个设备运行优化器步骤来更新其参数副本——从优化器的角度来看，它只是在优化一个本地模型。参数和优化器状态将在所有不同设备上保持同步，因为它们都从相同的初始模型和优化器状态开始，并且每个迭代都使用相同的平均梯度。此时，我们已完成一次训练迭代，可以重复该过程。

问题 (naive_ddp)：5 分

交付物：编写一个脚本，在反向传递后通过全归约来执行简单的分布式数据并行训练。为了验证你的 DDP 实现的正确性，使用它在一个随机生成的数据的小玩具模型上进行训练，并验证其权重与单进程训练的结果是否匹配。[a]

[a] 如果你在写这个测试时遇到困难，查看 `tests/test_ddp_individual_parameters.py` 可能会有帮助

问题 (naive_ddp_benchmarking): 3 分

在这个简单的 DDP 实现中，参数在每次反向传播后会在所有排名之间进行独立的全归约。为了更好地理解数据并行训练的开销，创建一个脚本来测试你之前实现的语言模型，当使用这个简单的 DDP 实现进行训练时。测量每个训练步骤的总时间以及用于通信梯度的时间比例。在单节点设置（1 个节点 x 2 个 GPU）下收集测量数据，该节点大小如 §1.1.2 中所述的 XL 模型大小。

可交付成果：关于您的基准测试设置的描述，以及每轮训练的测量时间

迭代和为每个设置通信梯度所花费的时间。

2.3 改进最小DDP实现

我们在§2.2中看到的最低级DDP实现有几个关键限制：

1. 它为每个参数张量执行单独的全归约操作。每次通信调用都会产生开销，因此批量通信调用可能有利于最小化这种开销。
2. 它等待反向传播完成后再通信梯度。然而，反向传播是增量计算的。因此，当参数梯度准备好时，可以立即通信，而无需等待其他参数的梯度。这使我们能够将梯度通信与反向传播的计算重叠，从而减少分布式数据并行训练的开销。

在本部分的作业中，我们将逐一解决这些限制，并衡量对训练速度的影响。

2.3.1 减少通信调用次数

与其为每个参数张量发出通信调用，不如看看我们能否通过批处理 all-reduce 来提升性能。具体来说，我们将要 all-reduce 的梯度收集起来，将它们连接成一个单一的张量，然后在所有 rank 上对组合后的梯度进行 all-reduce。使用 `torch. utils. flatten dense tensors` 和 `torch. utils. unflatten dense tensors`可能会有所帮助。

- - - - - - - -

问题 (`minimal_ddp_flat_benchmarking`)：2分

修改你的最小 DDP 实现，使其能够通信所有参数的扁平化梯度张量。在之前使用的条件下（1个节点 x 2个GPU，模型大小如§1.1.2所述），将其性能与为每个参数张量发出 all-reduce 的最小 DDP 实现进行比较。

交付物：在分布式数据并行训练中，使用单个批处理 all-reduce 调用测量每个训练迭代的耗时以及梯度通信所花费的时间。1-2句话比较批处理通信与单独通信梯度的结果。

2.3.2 带参数梯度通信的重叠计算

虽然批量处理通信调用可能有助于降低执行大量小型all-reduce操作的开销，但所有通信时间仍然直接导致开销。为了解决这个问题，我们可以利用观察到的现象：反向传播会逐层增量计算梯度（从损失开始向输入方向移动）—因此，我们可以在梯度准备好后立即进行all-reduce，通过重叠反向传播的计算与梯度通信来降低数据并行训练的开销。

我们将从实现和基准测试一个分布式数据并行包装器开始，该包装器在反向传播过程中异步 all-reduce单个参数张量。以下指针可能有用：

反向钩子 要在参数的梯度在反向传播中累积后自动调用一个函数，你可以使用 `register_post_accumulate_grad_hook` 函数。[4]

[4]请参阅pytorch.org/docs/stable/generated/torch.Tensor.register_post_accumulate_grad_hook.html获取更多信息和使用示例。

异步通信 所有 PyTorch 集体通信操作都支持同步 (`async_op=False`) 和异步执行 (`async_op=True`)。同步调用会阻塞，直到集体操作被排队到 GPU 上。这并不意味着 CUDA 操作已完成，因为 CUDA 操作是异步的。话虽如此，后续使用输出的函数调用将按预期工作。[5] 相比之下，异步调用会返回一个分布式请求句柄——因此，当函数返回时，集体通信操作不保证已被排队到 GPU 上，更不用说完成了。要等待操作被排队到 GPU 上（从而让输出在后续操作中可用），你可以在返回的通信句柄上调用 `handle.wait()` 。

例如，以下两个示例均使用同步或异步调用对所有张量列表中的张量进行 all-reduce 操作：

```
tensors = [torch.rand(5) for _ in range(10)]

# Synchronous, block until operation is queued on GPU.
for tensor in tensor:
    dist.all_reduce(tensor, async_op=False)

# Asynchronous, return immediately after each call and
# wait on results at the end.
handles = []
for tensor in tensors:
    handle = dist.all_reduce(tensor, async_op=True)
    handles.append(handle)

# ...
# Possibly execute other commands that don't rely on the all_reduce results
# ...

# Ensure that all-reduce calls were queued and
# therefore other operations depending on the
# all-reduce output can be queued.
for handle in handles:
    handle.wait()
handles.clear()
```

问题 (`ddp_overlap_individual_parameters`)：5 分

实现一个 Python 类来处理分布式数据并行训练。该类应包装任意 PyTorch `nn.Module` ，并在训练前处理权重广播（以便所有进程具有相同的初始参数），并发出梯度平均的通信调用。我们建议以下公共接口：

`def __init__(self, module: torch.nn.Module)`：给定一个待并行化的已实例化 PyTorch `nn.Module`：构建一个 DDP 容器来处理跨进程的梯度同步。

`def forward(self, *inputs, **kwargs)`：调用包装模块的 `forward()` 方法，并传入提供的位置参数和关键字参数。

`def finish_gradient_synchronization(self)`：被调用时，等待异步通信

[5]在高级情况下，如果你使用多个CUDA流，可能需要跨流显式同步以确保输出为后续操作准备好。参见 pytorch.org/docs/stable/notes/cuda.html#cuda-streams。

调用需在GPU上排队。

要使用此类执行分布式训练，我们将向其传递一个要封装的模块，然后在运行 `optimizer.step()` 之前调用 `finish_gradient_synchronization()`，以确保优化器步骤（该操作依赖于梯度）可能已入队：

```
model = ToyModel().to(device)
ddp_model = DDP(model)

for _ in range(train_steps):
    x, y = get_batch()
    logits = ddp_model(x)
    loss = loss_fn(logits, y)
    loss.backward()
    ddp_model.finish_gradient_synchronization()
    optimizer.step()
```

交付物：实现一个容器类来处理分布式数据并行训练。该类应重叠梯度通信和反向传播的计算。要测试您的DDP类，首先实现适配器 [adapters.get_ddp_individual_parameters] 和 [adapters.ddp_individual_parameters_on_after_backward]（后者是可选的，取决于您的实现您可能不需要它）。

然后，要执行测试，运行 `uv run pytest tests/test_ddp_individual_parameters.py`。我们建议多次运行测试（例如，5次）以确保它可靠地通过。

问题 (ddp_overlap_individual_parameters_benchmarking)：1分

(a) 测试您的DDP实现在与单个参数梯度的通信重叠反向传播计算时的性能。将其性能与先前研究的设置进行比较（最简DDP实现，要么为每个参数张量发出all-reduce，要么在所有参数张量的连接上发出单个all-reduce），使用相同的设置：1个节点、2个GPU以及§1.1.2中描述的XL模型大小。

交付物：在反向传播与单个参数梯度通信重叠时，每轮训练迭代测量的时间，用1-2句话比较结果。

(b) 使用1节点、2个GPU、XL模型大小设置，用Nsight分析器分析您的基准测试代码，比较初始DDP实现与这个反向计算和通信重叠的DDP实现。可视化比较两个跟踪，并提供一个分析器截图，证明一个实现将计算与通信重叠，而另一个没有。

交付物：2张截图（一张来自初始DDP实现，另一张来自这个反向计算与通信重叠的DDP实现），直观展示通信是否与反向传播重叠。

2.3.3 桶状参数梯度的计算与通信重叠

在上一节（§2.3.2）中，我们将反向传播计算与各个参数梯度的通信重叠了起来。然而，我们之前观察到，批量通信调用可以提高性能，尤其是在我们有大量参数张量（这在深度Transformer模型中很常见）的情况下。我们之前的批量尝试一次性发送所有梯度，这需要等待反向传播过程完成。

在本节中，我们将尝试兼顾两者的优点，通过将参数组织成桶（减少总通信调用次数），并在每个桶的各个张量都准备好时进行全归约（使我们能够将通信与计算重叠）。

问题 (ddp_overlap_bucketed)：8分

实现一个Python类来处理分布式数据并行训练，使用梯度桶来提高通信效率。该类应包装任意的输入PyTorch `nn.Module`，并在训练前处理权重广播（以便所有进程具有相同的初始参数），并发出用于梯度平均的桶化通信调用。我们建议以下接口：

`def __init__(self, module: torch.nn.Module, bucket_size_mb: float):` Given an instantiated 待并行的PyTorch `nn.Module`，构建一个DDP容器来处理跨进程的梯度同步。梯度同步应桶化处理，每个桶最多包含 `bucket_size_mb` 的参数。

`def forward(self, *inputs, **kwargs):` 调用包装模块的 `forward()` 方法，并传入提供的位置参数和关键字参数。

`def finish_gradient_synchronization(self):` 调用时，等待异步通信调用需在GPU上排队。

除了添加 `bucket_size_mb` 初始化参数外，此公共接口与我们的先前DDP实现接口匹配，该实现会逐个通信每个参数。我们建议使用 `model.parameters()`的逆序将参数分配到桶中，因为在反向传播过程中梯度将大致按此顺序准备好。

交付物：实现一个容器类来处理分布式数据并行训练。该类应重叠梯度通信和反向传播的计算。梯度通信应分桶处理，以减少总通信调用次数。为测试你的实现，完成 [adapters.get_ddp_bucketed]、[adapters.ddp_bucketed_on_after_backward]和 [adapters.ddp_bucketed_on_train_batch_start]（后两者是可选的，取决于你的实现你可能不需要它们）。

然后，要执行测试，运行 `pytest tests/test_ddp.py`。 我们建议运行测试多次（例如，5次）以确保其可靠通过。

问题 (ddp_bucketed_benchmarking)：3分

(a) 使用与先前实验相同的配置（1个节点、2个GPU、XL模型大小）对分桶DDP实现进行基准测试，并改变最大桶大小（1、10、100、1000 MB）。

将你的结果与先前未分桶的实验进行比较——结果是否符合你的预期？如果不符合，为什么？你可能需要使用PyTorch分析器来更好地理解通信调用是如何排序和/或执行的。你预期实验设置中的哪些变化会产生与预期一致的结果？

交付物：各种桶大小的每轮训练时间测量值。3-4句话。 ence com-
对结果、预期以及任何不匹配的潜在原因进行评论。

(b) 假设计算一个桶的梯度所需时间与通信梯度桶所需时间相同。写出一个方程，将DDP的通信开销（即反向传播后花费的额外时间）建模为一个函数。

模型参数的总大小（字节）(s)、全归约算法带宽(w，计算为每个rank的数据大小除以完成全归约所需时间)、每次通信调用的开销（秒）(o)以及桶的数量(n_b)。根据这个公式，编写一个最小化DDP开销的最优桶大小的公式。

可交付成果：一个用于建模 DDP 开销的公式，以及一个用于最优桶大小的公式。

2.4 四维并行

虽然实现上更为复杂，但我们可以在更多维度上并行化我们的训练过程。最常见的是我们讨论 5 种并行方法：

- 数据并行 (DP) — 数据批次被分配到多个设备上，每个设备计算各自批次的梯度。这些梯度必须以某种方式在设备之间进行平均。
- 全分片数据并行 (FSDP) — 优化器状态、梯度和权重被分配到多个设备上。如果我们只使用 DP 和 FSDP，每个设备都需要在执行前向或反向传递之前从所有其他设备收集权重分片。
- 张量并行（TP）— 激活值会在一个新的维度上分片，每个设备计算其自身分片的结果。通过张量并行，我们可以沿输入或输出操作的分片维度进行分片。如果我们将权重和激活值沿对应维度分片，张量并行可以与FSDP有效结合使用。
- 流水线并行（PP）— 模型按层逐级拆分为多个阶段，每个阶段运行在不同的设备上。
- 专家并行（EP）— 我们将专家（在专家混合模型中）分配到不同的设备上，每个设备计算其自身专家的结果。

通常，我们总是将FSDP和TP结合使用，因此可以将它们视为一个并行轴。这给我们留下了4个并行轴：DP、FSDP/TP、PP和EP。我们也将专注于密集模型（非MoE），因此不会进一步讨论EP。

在分布式训练的推理中，我们通常将我们的集群描述为设备网格，网格的轴是我们定义并行性的轴。例如，如果我们有 16 个 GPU，并且模型远大于单个设备可以容纳的大小，我们可能会倾向于将网格组织成 4×4 个 GPU 的网格，其中第一个维度表示 DP，第二个维度表示组合的 FSDP 和 TP。

有关这些方法的工作原理以及如何推导它们的通信和内存成本的更多详细信息，请参阅 TPU 扩展手册第 5 部分的概述（Austin 等人 [2025]）。关于更详细的流水线并行讨论，请参阅《超规模攻略附录》（Nouamane Tazi [2025]）。本书的其余部分也包含大量其他可能对你有用的信息。

问题 (`communication_accounting`)：10 分

考虑一个新的模型配置，XXL，包含 `d_model=16384`、 `d_ff=53248` 和 `num_blocks=126`。由于对于非常大的模型，绝大多数的 FLOPs 都在前馈网络中，因此我们做出一些简化的假设。首先，我们省略了注意力机制、输入嵌入和输出线性层。
然后，我们假设每个 FFN 只是两个线性层（忽略激活函数），其中第一个的输入大小为 `d_model`，输出大小为 `d_ff`，第二个的输入大小为 `d_ff`，输出size `d_model`。 您的模型由 `num_blocks` 个这些线性层的块组成。 不要做任何激活检查点，并保持您的激活和梯度通信在 BF16 中，而您的累积梯度、主权重和优化器状态应在 FP32 中。

(a) 在单个设备上以FP32格式存储主模型权重、累积梯度和优化器状态需要多少内存？反向传播（这些将在BF16格式中）能节省多少内存？这相当于多少台H100 80GB GPU的内存容量？

交付成果： 请提供计算过程和一句话的答案。

(b) 现在假设你的主模型权重、优化器状态、梯度和一半的激活值（实践中每两层一个）分布在 N_{FSDP} 个设备上。请给出每个设备所需的内存表达式。 N_{FSDP} 的值需要是多少才能使总内存成本低于1 v5p TPU（每个设备95GB）？交付物：计算过程和一句话的答案。

(c) 仅考虑前向传播。使用 $W_{\text{ici}} = 2 \cdot 9 \cdot 10^{10}$ 的通信带宽和 $W_{\text{hbm}} = 1.2 \cdot 10^{12}$，根据《缩放指南》的符号，使用手册中所述。$M_Y = 1$ (3D网格)，其中 $X = 16$ 是您的FSDP维度， $Y = 4$ 是您的TP维度。在什么单设备批大小下该模型受计算限制？在此设置中，整体批大小是多少？

交付物： 您的计算结果和一句话的回应。

(d) 实际上，我们希望整体批大小尽可能小，并且始终有效利用计算资源（换句话说，我们希望永远不被通信限制）。我们还能采用哪些技巧来减少模型的批大小但保持高吞吐量？

可交付成果：一段回应。用参考文献和/或公式来支持你的主张。

3 优化器状态分片

分布式数据并行训练在概念上很简单，而且通常非常有效，但需要每个进程持有模型参数和优化器状态的独立副本。这种冗余可能带来显著的内存成本。例如，AdamW优化器每个参数维护两个浮点数，这意味着它的内存消耗是模型权重的两倍。Rajbhandari等人 [2020] 描述了几种通过跨进程分区（1）优化器状态、（2）梯度以及（3）参数来减少数据并行训练中冗余的方法，并在必要时在工作者之间进行通信。

在本作业的这一部分，我们将通过实现优化器状态分片的简化版本来减少每个秩的内存消耗。与其保存所有参数的优化器状态，每个秩的优化器实例将仅处理参数的一个子集（大约为 1 / `world_size`）。当每个秩的优化器执行优化步骤时，它将仅更新其分片中的模型参数子集。然后，每个秩将向其他秩广播其更新的参数，以确保模型参数在每次优化步骤后保持同步。

问题 (`optimizer_state_sharding`)：15 分

实现一个 Python 类来处理优化器状态分片。该类应包装任意的输入 PyTorch `optim.Optimizer`，并负责在每次优化步骤后同步更新的参数。我们建议以下公共接口：

`def __init__(self, params, optimizer_cls: Type[Optimizer], **kwargs: Any):` 初始化分片状态优化器。 `params` 是一个要优化的参数集合（如果用户希望对不同模型部分使用不同的超参数，例如学习率，则可以是参数组）；这些参数将在所有秩之间分片。 `optimizer_cls`参数指定要包装的优化器类型（例如， `optim.AdamW`）。最后，任何剩余的关键字参数都将传递给 `optimizer_cls` 的构造函数。确保在此方法中调用 `torch.optim.Optimizer` 超类构造函数。

`def step(self, closure, **kwargs):` 调用包装优化器的 `step()` 方法，并提供闭包和关键字参数。更新参数后，与其他 rank 同步。

`def add_param_group(self, param_group: dict[str, Any]):` 此方法应将参数组添加到分片优化器中。这是在分片优化器的超类构造函数构建期间调用的，也可能在训练期间调用（例如，用于逐步解冻模型中的层）。因此，此方法应处理在 rank 之间分配模型的参数。

交付物：实现一个容器类来处理优化器状态分片。要测试您的分片优化器，首先实现适配器 `[adapters.get_sharded_optimizer]`。然后，执行测试。
运行 `uv run pytest tests/test_sharded_optimizer.py`。 我们建议多次运行测试（例如，5 次）以确保它可靠地通过。

现在我们已经实现了优化器状态分片，让我们分析它对训练期间峰值内存使用的影响及其运行时开销。

问题 (`optimizer_state_sharding_accounting`)：5分

(a) 创建脚本以分析训练语言模型时（带和不带优化器状态分片）的峰值内存使用情况。使用标准配置（1个节点，2个GPU，XL模型大小），

报告模型初始化后的峰值内存使用情况，优化器步骤直接之前的内存使用，以及优化器步骤直接之后的内存使用。结果是否符合您的预期？分解每个设置中的内存使用情况（例如，参数内存占用多少，优化器状态占用多少等）。

交付物：2-3句回应，包含峰值内存使用结果，以及内存如何在不同的模型和优化器组件之间分配的分解说明。

(b) 我们的优化器状态分片实现如何影响训练速度？使用标准配置（1个节点、2个GPU、XL模型大小）测量启用和禁用优化器状态分片时的每轮迭代耗时。

交付物：2-3句回应，包含你的时间记录。

(c) 我们的优化器状态分片方法与ZeRO阶段1（在Rajbhandari等人2020年的论文中描述为ZeRO-DP P_{os}）有何不同？

可交付成果：2-3句总结任何差异，特别是与内存和通信量相关的差异。

第4章 尾声

恭喜你完成作业！希望你觉得这门作业有趣且有益，并学到了一些关于如何通过提高单GPU速度和/或利用多GPU来加速语言模型训练的知识。

参考文献

Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023. URL https://arxiv.org/abs/2307.08691.

Tri Dao, Daniel Y Fu, Stefano Ermon, Atri Rudra, and Christopher Re. Flashattention: Fast and memory-efficient exact attention with IO-awareness. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022. URL https://openreview.net/forum?id=H4DqfPSibmx.

Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax, 2018. URL https://arxiv.org/abs/1805.02867.

Horace He. Making deep learning go brrrr from first principles. 2022. URL https://horace.io/brrr_intro.html.

Jacob Austin, Sholto Douglas, Roy Frostig, Anselm Levskaya, Charlie Chen, Sharad Vikram, Federico Lebron, Peter Choy, Vinay Ramasesh, Albert Webson, and Reiner Pope. How to scale your model. 2025. Retrieved from https://jax-ml.github.io/scaling-book/.

Haojun Zhao Phuc Nguyen Mohamed Mekkouri Leandro Werra Thomas Wolf Nouamane Tazi, Ferdinand Mom. The ultra-scale playbook: Training llms on gpu clusters, 2025.

Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory optimizations toward training trillion parameter models, 2020. arXiv:1910.02054.